Project3

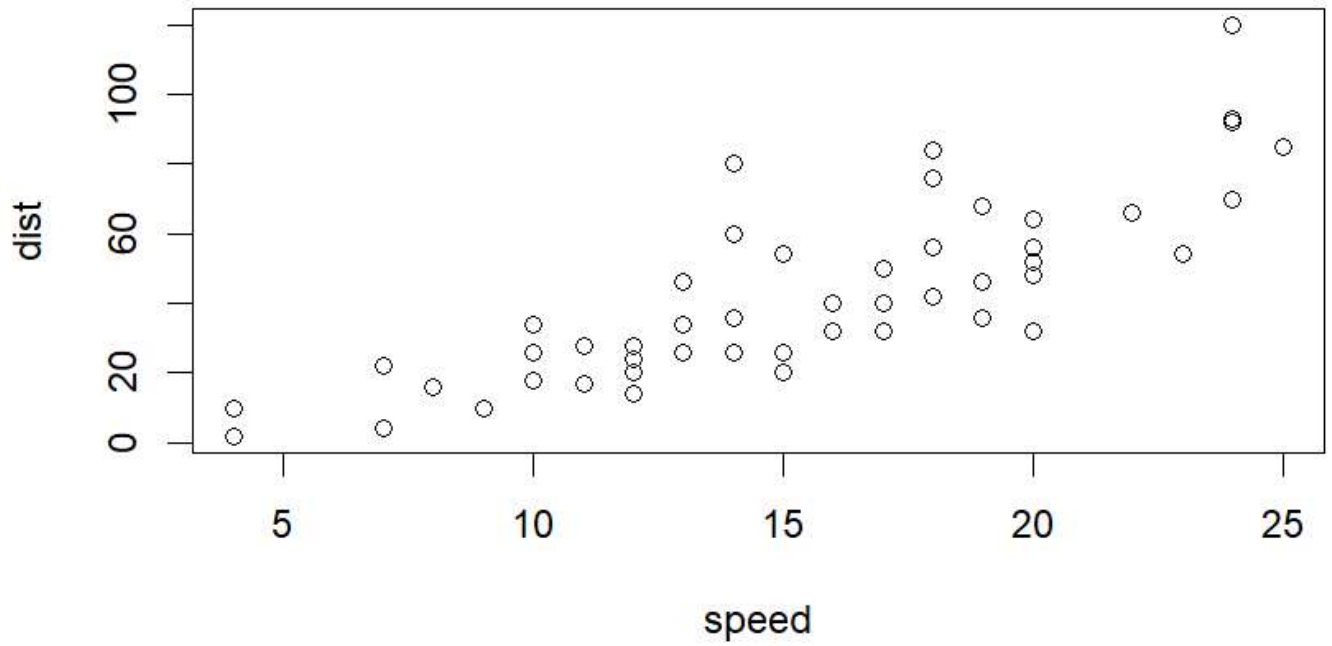
Code

This is an R Markdown (http://rmarkdown.rstudio.com) Notebook. When you execute code within the notebook, the results appear beneath the code.

Try executing this chunk by clicking the *Run* button within the chunk or by placing your cursor inside it and pressing *Ctrl+Shift+Enter*.

Hide

```
plot(cars)
```



Add a new chunk by clicking the *Insert Chunk* button on the toolbar or by pressing *Ctrl+Alt+I*.

When you save the notebook, an HTML file containing the code and output will be saved alongside it (click the *Preview* button or press *Ctrl+Shift+K* to preview the HTML file).

The preview shows you a rendered HTML copy of the contents of the editor. Consequently, unlike *Knit*, *Preview* does not run any R code chunks. Instead, the output of the chunk when it was last run in the editor is displayed.

Hide

```
# This is a small demonstartion of the command replicate(). It replicates drawing a sample of si
ze 15, 10 times
# from a normal distribution with mean 10 and variance 1. As you can see from the output, the sa
mples appear along
# the columns as the repetitions appear along the columns.

set.seed(1)
samples <- replicate(10, rnorm(15, 10, 1))
samples
```

```
          [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
 [1,]  9.373546  9.955066 11.358680  9.292505 12.401618 10.291446
 [2,] 10.183643  9.983810  9.897212 10.364582  9.960760  9.556708
 [3,]  9.164371 10.943836 10.387672 10.768533 10.689739 10.001105
 [4,] 11.595281 10.821221  9.946195  9.887654 10.028002 10.074341
 [5,] 10.329508 10.593901  8.622940 10.881108  9.256727  9.410479
 [6,]  9.179532 10.918977  9.585005 10.398106 10.188792  9.431331
 [7,] 10.487429 10.782136  9.605710  9.387974  8.195041  9.864821
 [8,] 10.738325 10.074565  9.940687 10.341120 11.465555 11.178087
 [9,] 10.575781  8.010648 11.100025  8.870637 10.153253  8.476433
[10,]  9.694612 10.619826 10.763176 11.433024 12.172612 10.593946
[11,] 11.511781  9.943871  9.835476 11.980400 10.475510 10.332950
[12,] 10.389843  9.844204  9.746638  9.632779  9.290054 11.063100
[13,]  9.378759  8.529248 10.696963  8.955865 10.610726  9.695816
[14,]  7.785300  9.521850 10.556663 10.569720  9.065902 10.370019
[15,] 11.124931 10.417942  9.311244  9.864945  8.746367 10.267099
          [,7]      [,8]      [,9]     [,10]
 [1,]  9.457480 11.767287  9.494043  8.463550
 [2,] 11.207868 10.716707 11.343039  9.699024
 [3,] 11.160403 10.910174  9.785421  9.471720
 [4,] 10.700214 10.384185  9.820443  9.347905
 [5,] 11.586833 11.682176  9.899809  9.943103
 [6,] 10.558486  9.364264 10.712666  8.085641
 [7,]  8.723408  9.538355  9.926436 11.176583
 [8,]  9.426735 11.432282  9.962366  8.335028
 [9,]  8.775387  9.349304  9.318340  9.536470
[10,]  9.526599  9.792619  9.675730  8.884080
[11,]  9.379633  9.607192 10.060160  9.249181
[12,] 10.042116  9.680007  9.411106 12.087167
[13,]  9.089078  9.720887 10.531496 10.017396
[14,] 10.158029 10.494188  8.481606  8.713699
[15,]  9.345415  9.822670 10.306558  8.359394
```

Hide

```
# We can cycle through the columns and calculate the average of each column as follows by loopin
g through
# with a for loop.


for(i in 1:10) {
    cat(mean(samples[,i]), "is the average for column", i, "\n")
}
```

```
10.10084 is the average for column 1
10.06407 is the average for column 2
10.09029 is the average for column 3
10.17526 is the average for column 4
10.18004 is the average for column 5
10.04051 is the average for column 6
9.942512 is the average for column 7
10.28415 is the average for column 8
9.915281 is the average for column 9
9.424663 is the average for column 10
```

Hide

```
# As an exercise read the documentation for the functions apply, sapply etc and see if you can r
ewrite
# the code to avoid the use of the for loop.
```

Hide

```
set.seed(1)
samples <- replicate(1000, rnorm(40, 10, 1))
sample_mean <- 1:1000
for(i in 1:1000) {
    sample_mean[i] <- mean(samples[,i])
}

#Empirical verification of unbiasedness
mean(sample_mean)
```

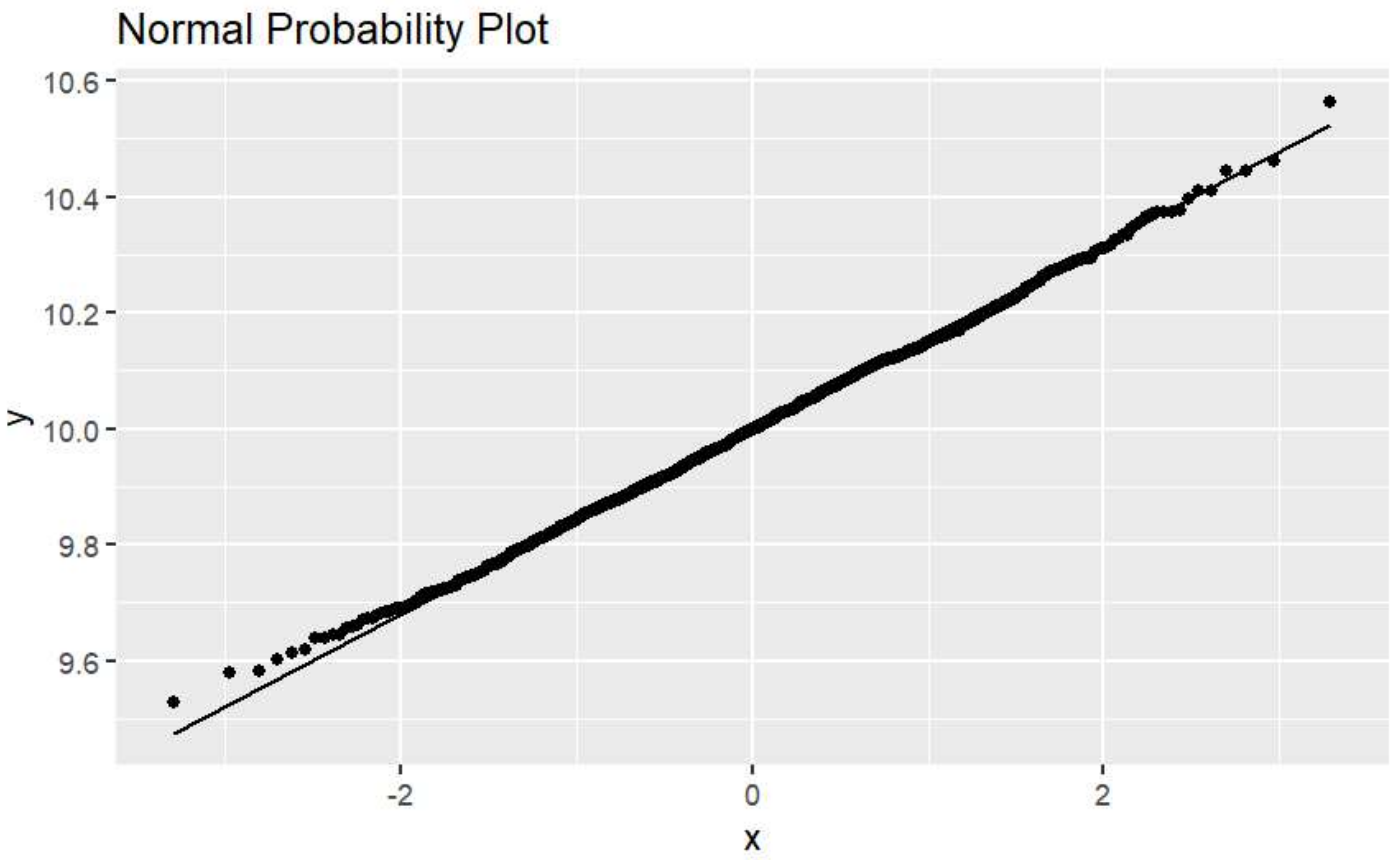
```
[1] 9.99782
```

Hide

```
#Normality plot

library(ggplot2)

# Create the normal probability plot
ggplot(data.frame(sample_mean), aes(sample = sample_mean)) +
stat_qq() +
stat_qq_line() +
labs(title = "Normal Probability Plot")
```



Question 1:

Hide

```
set.seed(67)

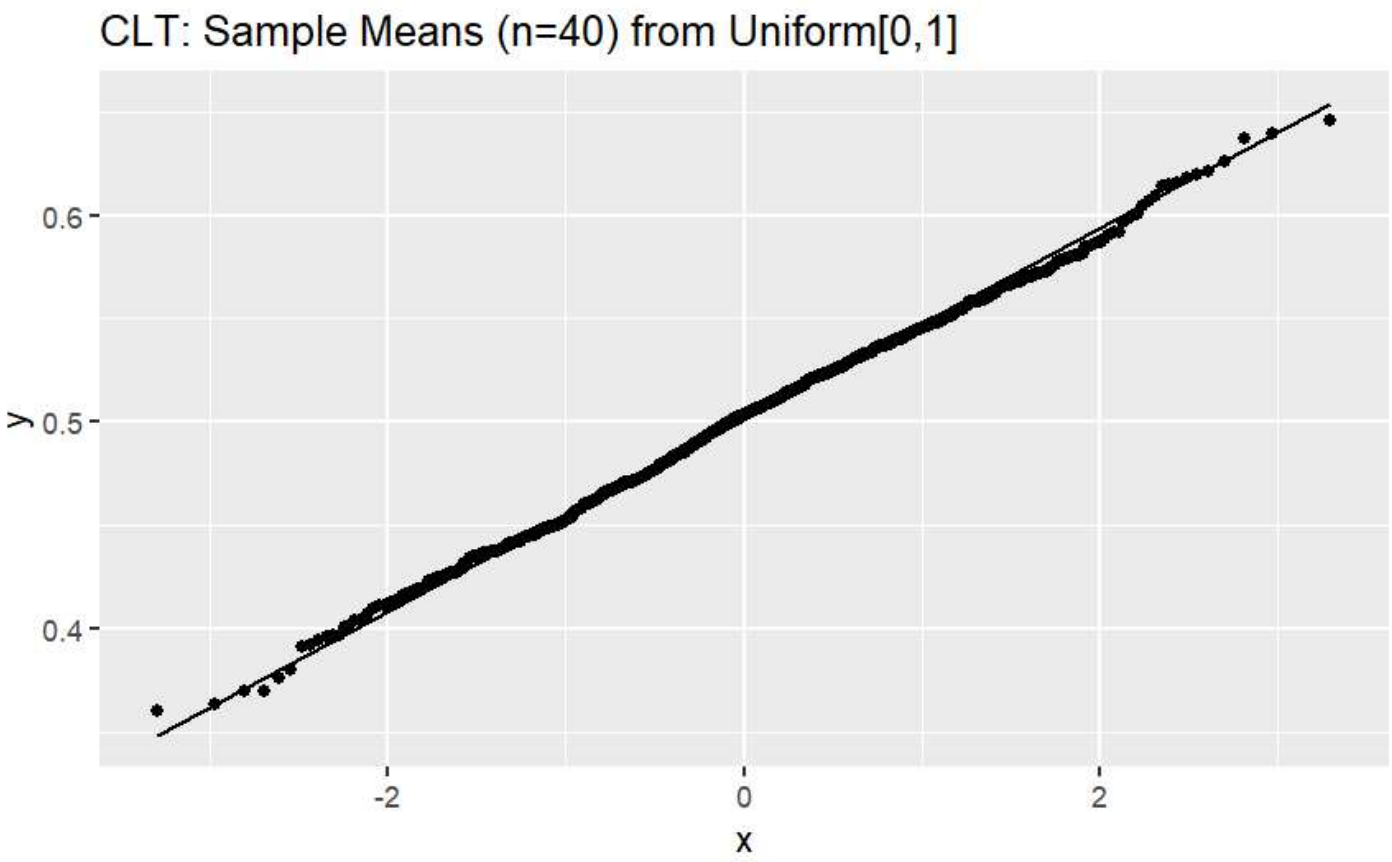
means_40 <- replicate(1000, mean(runif(40, 0, 1)))

mean(means_40)
```

```
[1] 0.500969
```

Hide

```
library(ggplot2)
ggplot(data.frame(means = means_40), aes(sample = means)) +
  stat_qq() + stat_qq_line() +
  labs(title = "CLT: Sample Means (n=40) from Uniform[0,1]")
```



Hide

```
set.seed(67)

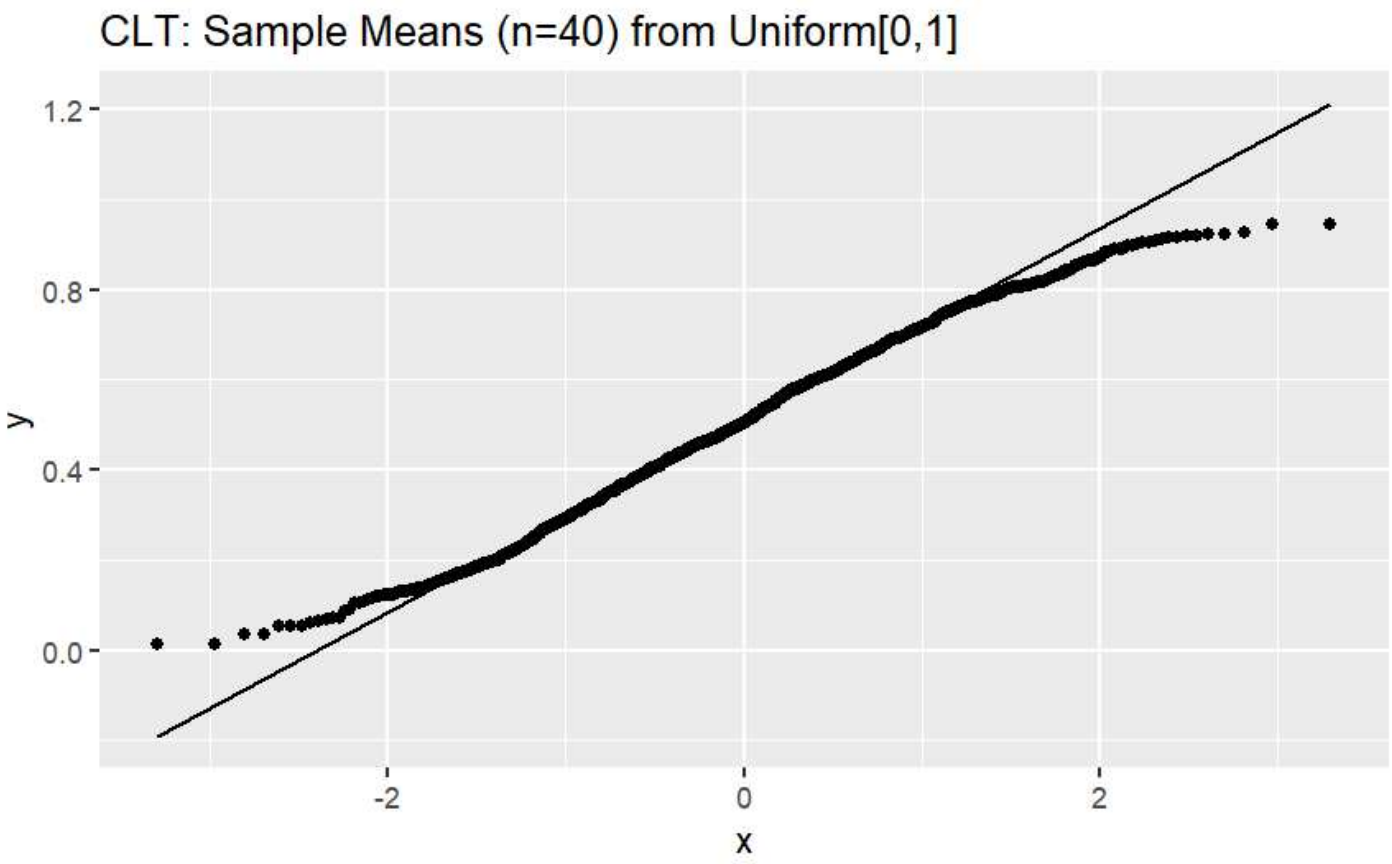
means_2 <- replicate(1000, mean(runif(2, 0, 1)))


mean(means_2)
```

```
[1] 0.5052997
```

Hide

```
library(ggplot2)
ggplot(data.frame(means = means_2), aes(sample = means)) +
  stat_qq() + stat_qq_line() +
  labs(title = "CLT: Sample Means (n=40) from Uniform[0,1]")
```



Hide

```
set.seed(67)

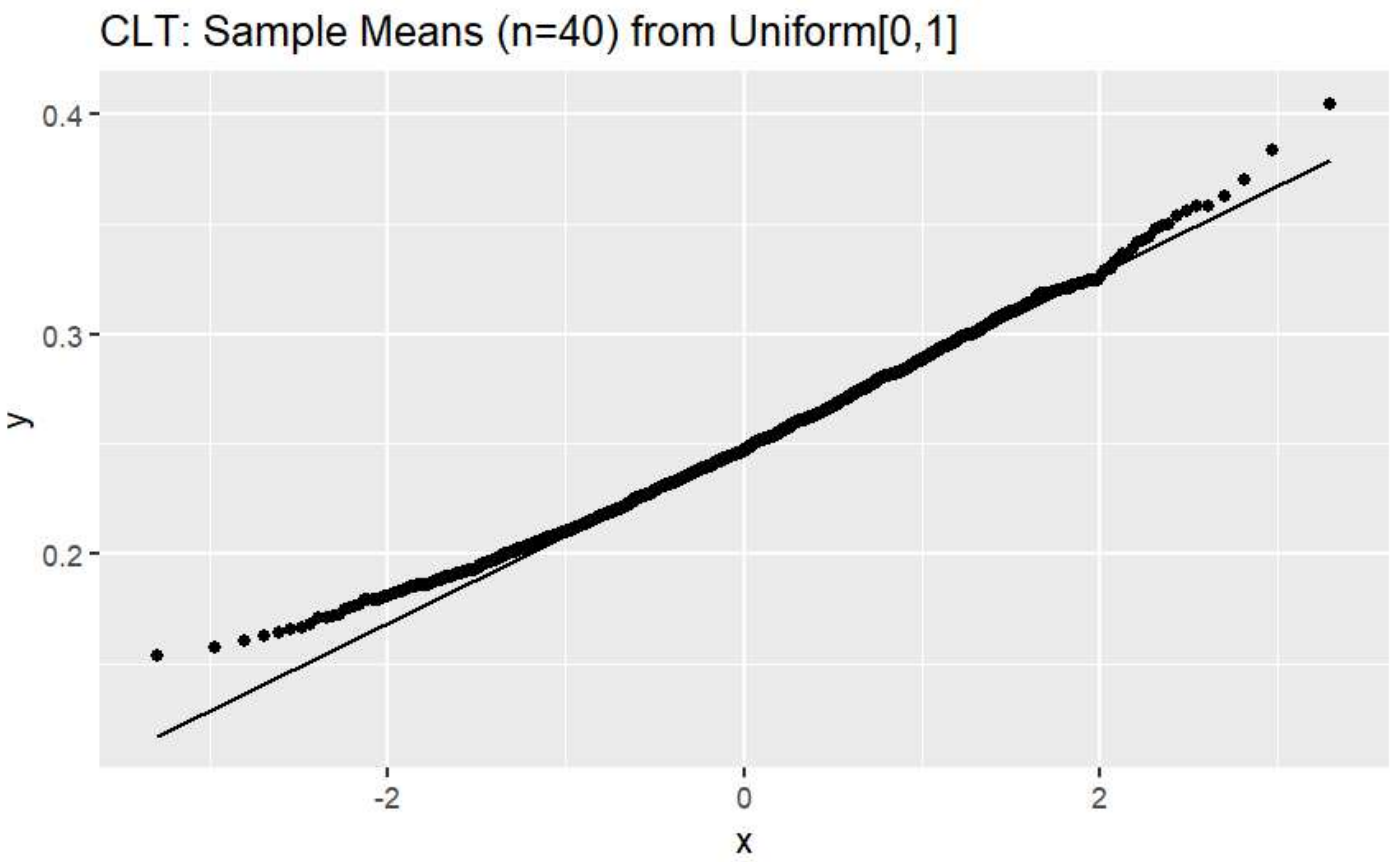
means_exp <- replicate(1000, mean(rexp(40, 4)))


mean(means_2)
```

```
[1] 0.5052997
```

Hide

```
library(ggplot2)
ggplot(data.frame(means = means_exp), aes(sample = means)) +
  stat_qq() + stat_qq_line() +
  labs(title = "CLT: Sample Means (n=40) from Uniform[0,1]")
```



Question 2: i) X~U[0,12] Var(X) = (12 - 0)^2/12 = 12 Estimator S = 1/3 sigma[(Xi - Xbar)^2] Bias[S] = E[S] - variance = -4

Hide

```
set.seed(67)
x <- runif(12000, 0, 12)

y <- var(x)
y
```

```
[1] 11.94501
```

Hide

```
'y should be near 12 which shows consistency'
```

```
[1] "y should be near 12 which shows consistency"
```

Hide

```
set.seed(67)

unb <- numeric(10)
b   <- numeric(10)

for(i in 1:10) {
  samp <- runif(3, 0, 12)
  unb[i] <- var(samp)
  b[i] <- mean((samp - mean(samp))^2)
}

data.frame(
  Unbiased = unb,
  Biased   = b
)
```

Unbiased <dbl>	Biased <dbl>
19.241166	12.8274438
20.764336	13.8428909
8.005942	5.3372946
12.851983	8.5679886
6.020093	4.0133951
18.394392	12.2629281
15.074900	10.0499336
24.331229	16.2208196
9.091090	6.0607269
1.429952	0.9533013

1-10 of 10 rows

Hide

```
mean(unb)
```

```
[1] 13.52051
```

Hide

```
mean(b)
```

```
[1] 9.013672
```

No they are not good estimators because the values are all over the place and not clustered around 12.

Hide

```
set.seed(67)

unb2 <- replicate(4000, var(runif(3, 0, 12)))
b2  <- replicate(4000, mean((runif(3,0,12)) - mean(runif(3,0,12)))^2)

unb2 <- mean(unb2)
b2 <- mean(b2)
cat("Unbiased", unb2, "\n")
```

```
Unbiased 11.97896
```

Hide

```
cat("Biased", b2)
```

```
Biased 7.707541
```

Yes my answers reflected my understanding as the unbiased estimator was close to 12 (12 * 2/(3-1)) and the biased estimator was close to 8 (12 * 2/3)

Question 3:

Hide

```
set.seed(67)

xmle <- rnorm(4000, 2, sqrt(5))

mlemu <- mean(xmle)
mlevar <- mean((xmle - mlemu)^2)

cat("mu_hat = ", mlemu, "\n")
```

```
mu_hat =  2.043616
```

Hide

```
cat("var_hat = ", mlevar)
```

```
var_hat =  4.864494
```